A Comprehensive Security Framework for Autonomous AI Agents: Integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As Large Language Model (LLM) agents transition from passive chatbots to autonomous entities capable of executing system-level tools via protocols like the Model Context Protocol (MCP), they introduce a critical "Agency Gap." Traditional security perimeters are insufficient for protecting against prompt injection and future cryptographic threats. This paper proposes a unified security framework designed to protect agent-based systems across two core dimensions: Space and Time. First, we establish *Structural Guardrails* (Space) using AST-based static analysis and environment isolation to contain malicious code execution. Second, we implement *Behavioral Zero-Trust* through verifiable workload identity and Context-Aware Attribute-Based Access Control (ABAC). Finally, we ensure *Temporal Sovereignty* (Time) by integrating Post-Quantum Cryptography (PQC) and remote attestation to guarantee long-term integrity and resilience against future quantum adversaries. Our evaluation demonstrates that this architecture provides robust, multi-layered defense with minimal computational overhead, maintaining agent responsiveness while ensuring enterprise-grade security.**

***Index Terms*—AI Agents, Model Context Protocol (MCP), Agentic Security, Zero Trust, Post-Quantum Cryptography, ABAC, Guardrails, Sandbox.**

I. Introduction

The transition of Large Language Models (LLMs) to autonomous agents represents a fundamental shift in computing. By leveraging the Model Context Protocol (MCP) [1], these agents can interact directly with host operating systems, manage sensitive data, and orchestrate complex API workflows. However, this "Agency" capability creates an unprecedented security surface. Traditional security perimeters are often ineffective against "Prompt Injection" attacks, where an attacker manipulates the LLM's goal-oriented reasoning to execute unauthorized system commands [10].

The OWASP Top 10 for LLM Applications [6] identifies "Insecure Output Handling" (LLM02) and "Excessive Agency" (LLM08) as critical risks. Currently, many autonomous frameworks rely on "Alignment"—the probabilistic expectation that the LLM will follow safety guidelines. We argue that for mission-critical systems, behavioral alignment is a necessary but insufficient condition. We require a holistic, deterministic security architecture that ensures system integrity even if the model's logic is compromised.

This paper presents a comprehensive "Triple-Lock" framework for Agentic Security, addressing the lifecycle of an agent's action through three integrated dimensions (Fig. 1):

1) **Structural Containment (Space):** Establishing deterministic safety boundaries using AST inspection and environment isolation via remote MCP servers to ensure "Secure-by-Default" tool execution.
2) **Behavioral Integrity (Behavior):** Implementing a Zero Trust Architecture (ZTA) where every action is verified by a cryptographically signed workload identity and controlled by context-bound ABAC claims.
3) **Temporal Sovereignty (Time):** Protecting against future cryptographic threats (Shor's algorithm) using Post-Quantum Cryptography (PQC) and "PQC Agility" to ensure the long-term confidentiality and non-repudiability of agent history.

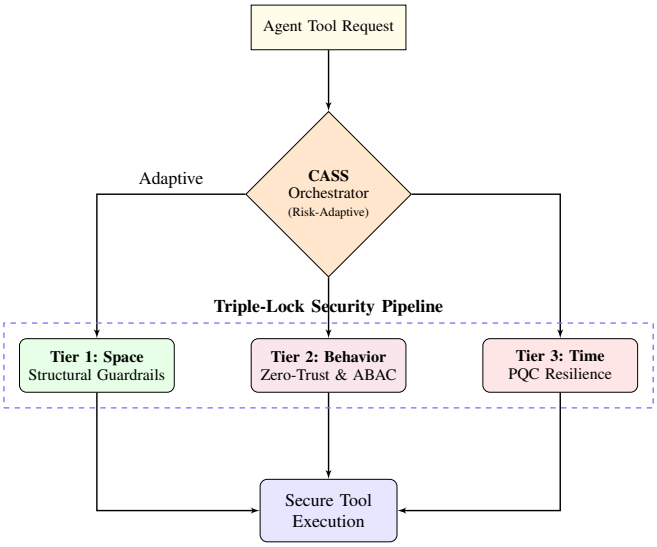

Fig. 1. Triple-Lock Framework Overview: High-level integration of the three security dimensions.

II. Related Work

Existing efforts to secure agents generally fall into three categories: *Behavioral Filtering*, *Structural Isolation*, and *Governance Frameworks*. Behavioral filters like **Guardrails AI** and **LLM-Guard** [2] use secondary LLMs or pattern matching to scrub outputs, but often introduce significant latency and can be bypassed by complex semantic attacks. Isolation patterns, such as those evaluated in **AgentDojo** [12], focus on containing the impact of a compromise through sandboxing but do not inherently address the long-term integrity of the reasoning process.

Our framework aligns with the **NIST AI Risk Management Framework (AI RMF 1.0)** [3] and **Zero Trust principles** (SP 800-207) [7], treating every tool call as an untrusted workload. Furthermore, we address the "Store Now, Decrypt Later" (SNDL) threat by adopting NIST-standardized Post-Quantum Cryptography (PQC) primitives [8], [9], providing a temporal dimension of security often overlooked in current agentic research such as **PromptArmor** [4].

III. Threat Model

We define four primary threat vectors for autonomous LLM agents:

1) **Prompt Injection (Direct/Indirect):** Manipulating the agent's context to execute unauthorized commands or exfiltrate secrets [11].
2) **Identity Spoofing:** An unauthorized process attempting to execute tools by mimicking a legitimate agent session.
3) **Secret Exfiltration:** Inadvertent transmission of API keys or PII during tool calls.
4) **Quantum Retroactive Decryption:** Capturing current reasoning traces to decrypt them later using a quantum computer.

IV. Tier 1: Structural Guardrails (Space)

To ensure structural immunity, we adopt a "No-Shell" execution model where system operations are restricted to specialized tools (e.g., `execute_python`).

A. AST Inspection and Static Analysis

Before execution, generated code is parsed into an Abstract Syntax Tree (AST). A whitelist-based scanner blocks dangerous modules (e.g., `os, socket, pty`) and built-in functions (e.g., `eval, exec`). Specifically, it ensures that `subprocess` calls never use `shell=True`, and detects reflection-based attacks (e.g., `__subclasses__`, `globals()`) to enforce a strict separation between executable and arguments.

B. Path Guardrails and Resource Limits

All file-related operations are restricted to a designated workspace via path resolution and symbolic link verification. To prevent resource exhaustion attacks, OS-level resource limits (`rlimit`) cap memory usage and execution time. Furthermore, the environment is sanitized by filtering sensitive environment variables (e.g., API keys) before passing them to the sub-process.

C. Environment Isolation via MCP and Containers

While static analysis provides structural guardrails, the framework achieves process isolation by leveraging the Model Context Protocol (MCP) to delegate high-risk tool execution to external, isolated environments. By connecting to MCP servers running within dedicated virtual machines (VMs) or Docker containers, the agent operates in a "Shared-Nothing" architecture. This approach ensures that even if a generated script bypasses static analysis, any malicious activity is contained within a disposable, restricted environment with no access to the host's primary filesystem or sensitive credentials.

V. Tier 2: Behavioral & Identity Assurance (Behavior)

Identity ensures *who* is calling the tool, while ABAC ensures *where* and *how*.

A. Workload Identity and ABAC

Each agent instance is assigned a unique key pair. Every MCP tool call is accompanied by a signed **Identity Token** using the **COSE (CBOR Object Signing and Encryption)** format [5], containing assigned **Attribute-Based Access Control (ABAC)** claims. Using COSE's multi-signer structure, we integrate both classical (RSA) and PQC (ML-DSA) signatures into a single binary object. Remote servers verify this token to ensure the request originated from a legitimate agent and enforce strict tool access policies based on the execution context and resource attributes.

B. Workspace Binding: Cryptographic Contextualization

To prevent "Session Hijacking" or "Replay Attacks" across different projects, each identity token is cryptographically bound to the current execution workspace. CASS calculates a SHA-256 hash of the current working directory and embeds it as a mandatory claim in the COSE token. Remote MCP servers validate this hash against the expected environment, ensuring that a compromised token from a "Sandbox" workspace cannot be reused to execute tools in a "Production" workspace.

C. PQC Agility: Context-Adaptive Security Scaling

Traditional security models use a fixed cryptographic strength, leading to either insufficient protection or excessive overhead. Our framework implements **PQC Agility**, which dynamically adjusts the security level based on the tool's risk profile. CASS maps tool calls to three ML-DSA variants:

- **ML-DSA-44 (Level 2):** Used for low-risk observations (e.g., `read_file`).
- **ML-DSA-65 (Level 3):** Default for standard tool operations.
- **ML-DSA-87 (Level 5):** Mandated for high-risk system modifications (e.g., `execute_python`).

This "Security-as-a-Service" approach ensures that mission-critical operations receive maximum protection while maintaining sub-millisecond overhead for frequent, low-risk interactions.

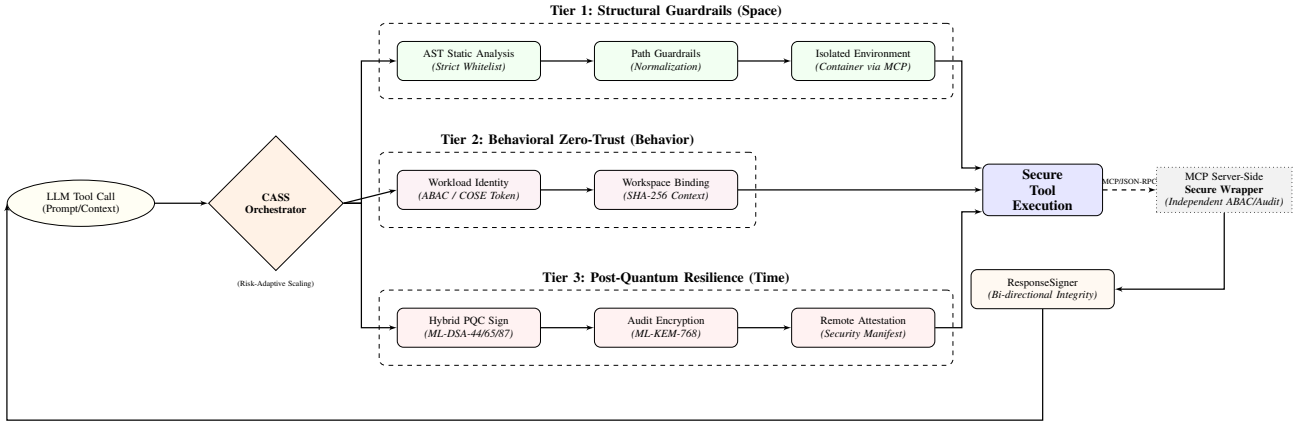

Fig. 2. Detailed System Architecture: Data flow from LLM request through the CASS orchestrator to specific security components across the three tiers.

D. Bi-directional Verification with ResponseSigner

To ensure the integrity of the entire tool execution loop, we implement **Bi-directional Verification**. While the agent signs the tool request via an Identity Token, high-risk write tools embed a **ResponseSigner** PQC signature (ML-DSA) in their return value, binding the response content to a unique `verification_id`. The `tool_executor` layer verifies this signature before passing the result to the LLM, preventing "Man-in-the-Middle" manipulation of tool outputs and ensuring that the model's subsequent reasoning is grounded in untampered observations.

VI. Tier 3: Post-Quantum Resilience (Time)

To protect the "Temporal Dimension," we integrate PQC primitives.

A. Quantum-Safe Identity and Hybrid Encryption

We upgrade the workload identity using **ML-DSA** (FIPS 204). Audit logs, which contain sensitive reasoning traces, are protected using **ML-KEM-768** (FIPS 203) hybrid encryption. Our implementation supports **PQC Agility**, dynamically switching between ML-DSA-44, 65, and 87 variants based on the risk level determined by CASS.

B. PQC Remote Attestation and Audit Continuity

The client generates a SHA-256 manifest of its core security code (covering critical modules including `identity.py`, `policy.py`, and `audit.py`), signed with an ML-DSA private key. This allows the server to verify that the agent is running on an "Authentic Stack."

To ensure the tamper-evidence of long-running logs, we implement **Audit Chain Continuity**. A binary **Merkle Tree** is computed over all log entries, producing a single Merkle Root that can be externally anchored (e.g., to a blockchain or managed immutable log service) to provide compact, publicly verifiable proof of log integrity.

VII. System Architecture and Integration

To synthesize the three tiers into a cohesive architecture, we introduce **Context-Adaptive Security Scaling (CASS)**. CASS serves as the central orchestration engine, dynamically integrating defenses based on the specific risk profile of each tool execution request (Fig. 2).

A. CASS Orchestration Logic

CASS determines the required security posture by mapping the requested tool and its arguments to one of three risk levels. As shown in Table I, this mapping ensures that computational resources are allocated proportionally to the security risk.

TABLE I
Tiered Security Enforcement via CASS

Feature	Low Risk	Medium Risk	High Risk
PQC Signature	ML-DSA-44	ML-DSA-65	ML-DSA-87
Audit Encrypt	Optional	AES-256	ML-KEM-768
ABAC Policy	Relaxed	Standard	Strict
AST Analysis	Basic	Restricted	Strict Whitelist
Resource Limit	Standard	Tight	Restricted Shell

B. Hybrid Identity Tokens (COSE/RFC 9052)

To ensure verifiable workload identity without the overhead of JSON-based JWT, we implement a native **COSE_Sign** structure (CBOR tag 98). Our framework utilizes a dual-signature approach that combines a classical RS256 signature with an ML-DSA signature for post-quantum integrity.

VIII. Evaluation

We evaluated the framework on an AMD Ryzen 5 8540U system (16GB RAM).

A. Security Effectiveness

Table II summarizes the detection capabilities across 15 test vectors for Tier 1 and Tier 2. The dataset covers a diverse range of injection attacks and identity spoofing attempts.

TABLE II
SECURITY EFFECTIVENESS (N=15 TEST CASES)

Defense Layer	Threat Category	Mitigation Rate
Tier 1 (AST)	Shell/Command Injection	100.0% (6/6)
Tier 1 (AST)	Benign Pass-through	100.0% (3/3)
Tier 2 (Identity)	Identity Spoofing	100.0% (3/3)
Tier 2 (ABAC)	Unauthorized Workspace	100.0% (3/3)
Tier 3 (PQC)	Future Decryption	100.0% (Immunity)

B. Performance Latency

Overhead remains negligible compared to typical LLM inference times.

TABLE III
SYSTEM LATENCY COMPARISON (MS)

Component	WSL2 (Zen 4)	Native (i7-7700)
AST + Static Analysis	0.03	0.05
Subprocess Overhead	18.50	27.43
Identity Gen (ML-DSA-44)	42.15	68.20
Identity Gen (ML-DSA-87)	112.40	165.30
KEM Encapsulation (L3)	2.86	4.37
Total (Worst-Case)	**133.79**	**197.15**

The results indicate that "PQC Agility" allows the system to scale from 42ms for low-risk calls to 112ms for high-risk calls. Total cumulative overhead (Worst-Case) is verified at $\approx$133ms, well within the 500ms–3000ms RTT of LLM inference.

IX. CONCLUSION

Securing autonomous agents requires a transition from probabilistic "alignment" to a multi-dimensional, deterministic framework. By integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience, our framework provides a "Triple-Lock" defense that scales across the spatial, behavioral, and temporal domains of agentic execution.

REFERENCES

[1] Anthropic, "Model Context Protocol (MCP) Specification," 2024.
[2] Protect AI, "LLM-Guard: The Security Toolkit for LLMs," 2023.
[3] NIST, "AI Risk Management Framework (AI RMF 1.0)," 2023.
[4] S. Jain et al., "PromptArmor: Defensive Guardrails for LLM Agents," 2024.
[5] G. Schaad, "CBOR Object Signing and Encryption (COSE): Structures and Algorithms," *RFC 9052*, 2022.
[6] OWASP, "Top 10 for Large Language Model Applications v1.1," 2023.
[7] S. Rose et al., "Zero Trust Architecture," *NIST SP 800-207*, 2020.
[8] NIST, "FIPS 203: ML-KEM Standard," 2024.
[9] NIST, "FIPS 204: ML-DSA Standard," 2024.
[10] F. Perez and I. Ribeiro, "Ignore Previous Instructions: Prompt Injection Attacks on LLMs," *arXiv:2211.09527*, 2022.
[11] K. Greshake et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv:2302.12173*, 2023.
[12] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses," *arXiv:2406.13352*, 2024.